College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID608001: Intermediate Application Development Concepts
Level 6, Credits 15

Project

Assessment Overview

In this **individual** assessment, you will design and develop **two** web applications.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Apply design patterns and programming principles using software development best practices.
2. Design and implement full-stack applications using industry relevant programming languages.

Assessments

Assessment	Weighting	Due Date	Learning Outcomes
Practical	20%	Sunday 9th November at 11.59 PM	1
Project	80%	Sunday 9th November at 11.59 PM	1 and 2

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Sunday 9th November** at **11.59 PM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** over all assessments in **ID608001: Intermediate Application Development Concepts**.

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcomes for this assessment.

Learning to use AI tools is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

It also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at https://www.op.ac.nz/about-us/governance-and-management/policies.

Submission

You **must** submit all files via **GitHub**. Create a repository and add the course lecturer as a collaborator. The latest files will be used to mark against the marking rubric. Please test before you submit. Partial marks **will not** be given for incomplete functionality. Late submissions will incur a **10% penalty per day**, rolling over at **12.00 AM**.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments **are not** applicable in **ID608001: Intermediate Application Development Concepts**.

Functionality - Learning Outcomes 1, 2 (25%)

- **HackerNews API** application with the following functionality:
  - Data is fetch from the HackerNews API.
  - Display a spinner while the data is being fetched from the API.
  - Responsive UI design using **Tailwind CSS**.
  - **Story Features:**
    - Display the 50 ask, best, job, new, show and top stories.
    - For each story, display the title, by, text, time and a link to the story using the id. Here is an example link - https://news.ycombinator.com/item?id=ID, where ID is the story's id.
    - Truncate the text to 100 characters and add a link that expands the text when clicked.
  - **Leader Features:**
    - Display 100 leaders. Here is a link to the leaders - https://news.ycombinator.com/leaders.
    - For each leader, display their id, karma, created, about and a link to their profile using the id. Here is an example link - https://news.ycombinator.com/user?id=ID, where ID is the user's id.
  - **Favourites Features:**
    - Display a heart icon next to the story or leader. Toggle the heart icon when clicked to add or remove the story or leader from favourites.
    - Display a list of favourite stories and leaders.
    - If no favourites are found, display a message indicating that no favourites were found.
  - **Search Features:**
    - Search for stories by title or type.
    - Search for leaders by id.
    - The search should be case-insensitive and return results that contain the search term anywhere in the title, type or id.
    - Display the search results in a list.
    - If no results are found, display a message indicating that no results were found.
  - **Sort Features:**
    - Sort stories by time, score or type.
    - Sort leaders by karma or created.
    - The default sort order for stories is time and for leaders is karma.
  - Display the time and created in a human-readable format.
  - Select any five **Medium Features** from the list below:
    - Hover effects on stories and leaders.
    - Toggle between colour themes.
    - Display the domain name from the story's URL. For example, if the story's URL is `https://example.com/story`, display `example.com`.
    - Display score-range badges with colours. For example, 0-100 is red, 101-500 is orange, 501-1000 is yellow, 1001-5000 is green and 5001+ is blue.
    - Display type badges with colours. For example, ask is blue, best is green, job is yellow, new is purple, show is orange and top is red.
    - Select and compare two leaders. Display their id, karma, created, about and a link to their profile using the id.
    - Display karma-range badges with colours for leaders. For example, 0-100 is red, 101-500 is orange, 501-1000 is yellow, 1001-5000 is green and 5001+ is blue.
    - For stories, toggle between list and grid view. The list view should display the title, by, text, time and a link to the story using the id. The grid view should display the title, by and a link to the story using the id.
  - Select any two **Hard Features** from the list below:

    - Display the stories and leaders in pages of 10 with ”Previous” and ”Next” navigation buttons.
    - Display stories and leaders search history based on the search functionality.
    - Display a graph of the karma for each leader. Display their id on the x-axis and karma on the y-axis.
    - Display a list of recommended stories based on the user’s favourites. The recommendations should be based on the user’s favourites and the stories’ type.

- **OpenTDB API** application with the following functionality:
    - Data is fetch from the https://opentdb.com/api_config.php.
    - Display a spinner while the data is being fetched from the API.
    - Responsive UI design using **Shadcn UI**.
    - Create a new quiz by selecting the start date, end date, number of questions, category, difficulty and type.
    - Display a list of past, current and future quizzes.
    - Past and future quizzes are not playable.
    - Play a quiz by selecting a current quiz from the list. The quiz should display the start date, end date, category, difficulty, type, questions, options and a submit button.
    - Shuffle the questions and options for each quiz.
    - Display the quiz results after submitting the quiz. Prompt the user to enter their name before displaying the results. The results should display the user’s name, start date, end date, category, difficulty, type, questions, options, correct answer and score.
    - Store the quiz results appropriately.
    - Display the average score for each quiz.
    - Display the total number of quizzes played.
    - For each quiz, display the scores in a list.
    - Select any three **Medium Features** from the list below:
        - Display a progress bar and ”Question X of Y” text while playing the quiz.
        - Display category badges with colours. For example, general knowledge is blue, art is green, history is yellow, geography is purple, etc.
        - Allow a user to retake a completed quiz.
        - For each quiz, display a countdown timer per question. The timer should start at ten minutes. If the timer reaches 0, the quiz should automatically submit.
        - Select and compare two users. Display their name, number of quizzes played and average score.
- Deploy both applications to **Vercel**.

Code Quality and Best Practices - Learning Outcomes 1, 2 (40%)

- Write clean, readable, and maintainable code.
- Use modular, reusable components and avoid code duplication.
- Keep code simple and easy to understand.
- Optimise performance by managing resources efficiently.

Version Control - Learning Outcome 1 (5%)

- Regular commits are made throughout development, demonstrating meaningful progress.
- Commit messages are clear and descriptive.
- The repository shows a clean and logical commit history that reflects the development process of the applications.

Reflection - Learning Outcomes 1, 2 (5%)

- Write a short reflection (300-500 words) on your development process and learning throughout the project, including:
  - Challenges you faced and how you overcame them.
  - What you would do differently in future projects.
  - New skills, tools or practices you learned during the project.

Presentation - Learning Outcomes 1, 2 (5%)

- Record a 5-10 minute screen recording of your **HackerNews API** and **OpenTDB API** applications.
- Demonstrate the applications' features.
- Make sure the video is clear and easy to follow.
- Submit a link to the presentation in your repository's **README.md** file.

Marking Rubric

Functionality - Learning Outcomes 1, 2 (25%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
HackerNews API - Data Fetching and UI (2%)	Data fetched correctly from HackerNews API. Spinner displays during loading. Fully responsive UI using Tailwind CSS across all screen sizes.	Data fetched correctly. Spinner present. Responsive UI with minor issues on some screen sizes.	Data fetched but may have minor issues. Basic spinner implemented. Partially responsive UI.	Data fetching fails or incomplete. No spinner or non-functional. Poor or no responsive design.
HackerNews API - Story Features (3%)	All 50 stories displayed correctly for each type (ask, best, job, new, show, top). All required fields shown (title, by, text, time, link). Text truncation to 100 chars with functional expand link working perfectly.	Stories displayed for all types. Most required fields shown. Text truncation mostly functional with minor issues.	Stories displayed for most types. Some required fields shown. Basic text truncation present.	Multiple story types missing. Required fields missing. Text truncation non-functional.
HackerNews API - Leader Features (2%)	100 leaders displayed correctly. All required fields shown (id, karma, created, about, profile link). Profile links formatted correctly.	100 leaders displayed. Most required fields shown. Profile links mostly functional.	Leaders displayed but count may be incorrect. Some required fields shown. Basic profile links present.	Leaders missing or significantly incomplete. Required fields missing. Profile links non-functional.
HackerNews API - Favourites Features (2%)	Heart icon toggles correctly for stories and leaders. Favourite list displays all saved items accurately. Clear message shown when no favourites exist.	Heart icon functional with minor issues. Favourite list mostly accurate. Message displayed for no favourites.	Heart icon partially functional. Favourite list displays some items. Basic no-favourites message present.	Favourites feature non-functional or missing. Heart icon doesn't toggle. No message for empty favourites.
HackerNews API - Search Features (2%)	Search works for stories by title and type. Search works for leaders by id. Case-insensitive and returns partial matches. Clear message when no results found.	Search mostly functional for stories and leaders. Case-insensitive with minor issues. No results message present.	Basic search implemented. May have case-sensitivity issues or limited functionality. Some no-results handling.	Search non-functional or missing. No case-insensitive handling. No message for empty results.

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
HackerNews API - Sort Features (2%)	Stories sort correctly by time, score and type. Leaders sort correctly by karma and created. Default sort order correctly implemented (stories: time, leaders: karma).	Sorting mostly functional for stories and leaders. Default sort orders present with minor issues.	Basic sorting implemented for some criteria. Default sort orders partially implemented.	Sorting non-functional or missing. Default sort orders not implemented.
HackerNews API - Time Formatting and Medium Features (3%)	Time and created displayed in human-readable format. Five medium features fully implemented and functional. Features well-integrated and polished.	Time formatting mostly correct. 4-5 medium features implemented with minor issues.	Basic time formatting present. 3 medium features implemented with some functionality gaps.	Time formatting missing or incorrect. Fewer than 3 medium features or non-functional.
HackerNews API - Hard Features (2%)	Two hard features fully implemented and functional. Features work correctly and are well-integrated.	Two hard features implemented with minor issues or limitations.	1-2 hard features partially implemented with functionality gaps.	Fewer than 1 hard feature or non-functional implementations.
OpenTDB API - Data Fetching and UI (2%)	Data fetched correctly from OpenTDB API. Spinner displays during loading. Fully responsive UI using Shadcn UI across all screen sizes.	Data fetched correctly. Spinner present. Responsive UI with minor issues on some screen sizes.	Data fetched but may have minor issues. Basic spinner implemented. Partially responsive UI.	Data fetching fails or incomplete. No spinner or non-functional. Poor or no responsive design.
OpenTDB API - Quiz Creation and Display (2%)	Quiz creation form with all fields (start date, end date, number of questions, category, difficulty, type). Past, current and future quizzes displayed correctly. Past and future quizzes correctly non-playable.	Quiz creation mostly functional with all fields. Quiz list displayed with minor issues. Playability restrictions mostly enforced.	Basic quiz creation with some fields. Quiz list partially functional. Some playability restrictions present.	Quiz creation missing fields or non-functional. Quiz list missing or incorrect. No playability restrictions.
OpenTDB API - Quiz Playing (2%)	Quiz playing fully functional with all required fields displayed. Questions and options shuffled correctly. Submit button works properly.	Quiz playing mostly functional. Shuffling works with minor issues. Submit button present.	Basic quiz playing implemented. Shuffling partially functional. Submit button present but may have issues.	Quiz playing non-functional or missing. No shuffling. Submit button missing or broken.

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
OpenTDB API - Results and Storage (2%)	Results display correctly after submission. Name prompt works. All required result fields shown. Quiz results stored appropriately and persistently.	Results mostly functional. Name prompt present. Most result fields shown. Results stored with minor issues.	Basic results display. Name prompt partially functional. Some result fields shown. Results storage incomplete.	Results non-functional or missing. No name prompt. Required fields missing. No results storage.
OpenTDB API - Statistics and Medium Features (1%)	Average score per quiz displayed correctly. Total quizzes played shown accurately. Score lists displayed for each quiz. Three medium features fully implemented.	Statistics mostly accurate. Score lists present. 2-3 medium features implemented with minor issues.	Basic statistics present. Score lists partially functional. 1-2 medium features implemented.	Statistics missing or incorrect. Score lists non-functional. Fewer than 1 medium feature.
Deployment (2%)	Both applications successfully deployed to Vercel. Applications are accessible, functional and perform well in production.	Both applications deployed to Vercel with minor issues or performance concerns.	Both applications deployed but with accessibility or functionality issues in production.	Applications not deployed, deployment non-functional, or only one application deployed.

Code Quality and Best Practices - Learning Outcomes 1, 2 (40%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Code Readability and Maintainability (10%)	Code is exceptionally clean and readable with consistent formatting. Meaningful variable and function names. Well-organised file structure. Comprehensive comments where needed.	Code is clean and readable with good naming conventions. Organised structure with some helpful comments. Minor inconsistencies.	Code is mostly readable. Some unclear naming or inconsistent formatting. Basic organisation present.	Code is difficult to read with poor naming conventions. Inconsistent or chaotic structure.
Modularity and Reusability (10%)	Excellent use of modular, reusable components. Functions and components are well-abstracted. No code duplication. DRY principles followed throughout.	Good modular structure with reusable components. Minimal code duplication. Most code follows DRY principles.	Some modular components present. Some code duplication exists. Partial adherence to DRY principles.	Little modularity. Significant code duplication. Poor component abstraction.
Code Simplicity and Clarity (10%)	Code is simple, elegant and easy to understand. Complex logic is well-explained. Appropriate use of language features without over-engineering.	Code is generally simple and clear. Most logic is easy to follow with minor complex sections.	Code works but contains unnecessarily complex sections. Some logic is difficult to follow.	Code is overly complex or convoluted. Difficult to understand the implementation.
Performance and Resource Management (10%)	Excellent resource management. Efficient API calls and data handling. Proper error handling and memory management. No performance bottlenecks. Optimised component rendering and state updates.	Good resource management. Mostly efficient API calls. Adequate error handling with room for improvement. Minor performance considerations.	Basic resource management. Some inefficient API calls or error handling gaps. Performance acceptable but not optimised.	Poor resource management. Inefficient API calls. Missing error handling. Performance issues present. Excessive re-renders or memory leaks.

Version Control - Learning Outcome 1 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Commit History and Messages (5%)	Regular, meaningful commits throughout development. Clear, descriptive commit messages. Logical progression showing development process. Clean commit history.	Regular commits with mostly clear messages. Commit history shows good development progression with minor gaps.	Commits present but irregular or clustered. Some commit messages unclear. Basic development history visible.	Few commits or poor timing. Unclear messages (e.g., ”update”, ”fix”). No clear development progression.

Reflection - Learning Outcomes 1, 2 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Reflection Quality (5%)	Thoughtful reflection (300-500 words) covering all required points. Clear articulation of challenges, solutions and learning. Demonstrates deep understanding and self-awareness.	Good reflection covering all required points. Adequate discussion of challenges and learning with some depth. Within word count.	Reflection present covering most points. Brief or superficial discussion. May be slightly outside word count.	Reflection missing, incomplete or significantly outside word count. Lacks depth or doesn’t cover required points.

Presentation - Learning Outcomes 1, 2 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Video Demonstration (5%)	Clear 5-10 minute recording demonstrating all HackerNews API and OpenTDB API features. Well-paced, professional presentation. Easy to follow with good audio/visual quality. Link submitted correctly.	Good recording within time limit. Most features demonstrated. Generally clear with minor audio/visual issues. Link submitted.	Recording present but may be too short/long. Some features demonstrated. Quality acceptable but could be clearer.	Recording missing, significantly outside time limit, unclear, or doesn’t demonstrate key features. Poor quality.